SIMATS ENGINEERING

CO2_ASSESSMENT TOOL2_Scenario Based Assignment

COURSE NAME: Artificial Intelligence COURSE CODE:CSA1709

Register Number	**192524317**
Name	**S DHANUSH**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 20%

QUESTIONS: 5 **Total Marks:50**

Code of Conduct:

I S DHANUSH (192524317)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
S DHANUSH
(192524317)

Scenario Based Assignment

1. Scenario: A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to:A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty

ii. Explain how A* would handle the same situation, including how it balances cost and heuristic

iii. Analyze the impact of heuristic accuracy on both algorithms

iv. Discuss how dynamic changes (blocked paths) affect search performance

v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

i. Greedy Best-First Search (GBFS): Behavior, Strengths, and Weaknesses

Working Principle:
Greedy Best-First Search selects the next node based only on the heuristic value **h(n)**, which estimates the remaining distance to the goal (e.g., straight-line distance). It always chooses the node that appears closest to the destination.

Behavior in the Given Scenario:

- The drone uses the straight-line distance to the destination to choose its path.
- It ignores the actual travel cost caused by flooded roads, bad weather, or difficult terrain.
- If a selected path becomes blocked, the algorithm must backtrack and search for another route.

Strengths:

- Very fast decision-making, which is useful during emergencies.
- Requires less computation than many other search algorithms.
- Quickly finds a feasible path when the heuristic is good.

Weaknesses under Uncertainty:

- Does not guarantee the shortest or safest route.
- Ignores changing movement costs such as weather and terrain.
- May repeatedly choose blocked or risky paths.
- Can become inefficient if the heuristic is misleading.
- Not optimal in dynamic environments

ii. A* Search: Cost and Heuristic Balance

Working Principle:
A* evaluates each node using the function:

$f(n)=g(n)+h(n)f(n) = g(n) + h(n)f(n)=g(n)+h(n)$

where:

- **g(n):** Actual cost from the start to the current node.
- **h(n):** Estimated cost from the current node to the goal.

Behavior in the Given Scenario:

- The drone considers both the actual travel cost and the estimated remaining distance.
- Variable movement costs due to terrain, wind, or rain are included in **g(n)**.
- When new information about blocked paths is received, A* recalculates and searches for a better alternative.
- The algorithm naturally avoids routes with high travel cost even if they appear geographically shorter.

Advantages:

- Finds the optimal route if the heuristic is admissible.
- Balances speed and path quality.
- Handles varying travel costs effectively.
- Produces safer and more reliable routes.

Limitations:

- Requires more memory and computation than Greedy Search.
- Frequent environmental changes may require repeated replanning.

iii. Impact of Heuristic Accuracy

Heuristic Accuracy	Greedy Best-First Search	A* Search
Highly Accurate	Very fast and often finds a good path	Fast and guarantees the optimal path
Moderately Accurate	May choose inefficient routes	Still generally performs well
Poor/Inaccurate	Performance degrades significantly; may explore wrong directions	Remains correct but expands more nodes, increasing computation

Analysis:

- **GBFS** depends almost entirely on the heuristic. A poor heuristic can lead to poor decisions.
- **A*** is less affected because it also considers the actual travel cost, making it more reliable.

iv. Effect of Dynamic Changes (Blocked Paths)

In a flood-affected region:

- Roads may become flooded unexpectedly.
- Weather conditions may suddenly increase travel time.
- Safe routes may become unsafe.

Effect on Greedy Best-First Search

- May repeatedly move toward blocked routes.
- Needs frequent backtracking.
- Performance decreases significantly in rapidly changing environments.

Effect on A*

- Updates path costs when new information is available.
- Can recompute a better route using both actual costs and heuristic estimates.
- More robust but incurs additional computational overhead during replanning.

v. Recommended Algorithm

Recommended Algorithm: A* Search

Justification:

Factor	Greedy Best-First Search	A* Search
Real-time speed	Excellent	Good
Path optimality	Not guaranteed	Guaranteed (with admissible heuristic)
Safety	Lower	Higher
Handles varying costs	Poor	Excellent
Handles blocked paths	Limited	Better with replanning
Reliability	Moderate	High

Reason:
Although Greedy Best-First Search is faster, it ignores actual travel costs and may direct the drone into flooded or dangerous routes. In emergency medicine delivery, **safety and reliable delivery are more important than saving a small amount of computation time**.

A* provides a balanced solution by considering both:

- the actual travel cost (terrain, weather, risk), and
- the estimated remaining distance.

This enables the drone to choose routes that minimize delay while avoiding unsafe or expensive paths.

Conclusion

For AI-powered emergency medicine delivery in a flood-affected region, **A*** is the most suitable algorithm. It balances actual travel cost with heuristic estimates, adapts better to changing conditions, and produces optimal, safe routes. While Greedy Best-First Search offers faster decisions, its reliance solely on the heuristic makes it less reliable in dynamic and uncertain environments where blocked paths and variable movement costs are common.

2. **Scenario**: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimizeTotal waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

i. Explain how Hill Climbing would work in this scenario and identify its limitations

ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation

iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations

iv. Recommend the best approach and justify your choice based on scalability and adaptability

i. Hill Climbing Algorithm

Working Principle

Hill Climbing is a local search algorithm that starts with an initial traffic signal configuration and repeatedly makes small changes that improve the objective function.

For example:

- Increase the green light duration at a busy intersection.
- Reduce the red light duration where traffic is low.
- Accept the change only if it reduces waiting time or congestion.

The process continues until no neighboring solution provides further improvement.

Advantages

- Simple and easy to implement.
- Requires very little memory.
- Produces quick improvements.

- Suitable for real-time optimization with small changes.

Limitations

- May become trapped in local optimal solutions.
- Cannot guarantee the best global solution.
- Highly dependent on the initial traffic signal configuration.
- Performs poorly in complex traffic networks.

ii. Local Maxima, Plateaus, and Ridges

1. Local Maximum

A local maximum is a solution that is better than all nearby solutions but is not the best overall.

Real-world Example:
The AI finds signal timings that reduce congestion in one neighborhood but increase traffic in nearby areas. Since no small adjustment improves the current solution, Hill Climbing stops even though a much better city-wide solution exists.

2. Plateau

A plateau is a flat region where many neighboring solutions have the same objective value.

Real-world Example:
Several different signal timing configurations produce nearly identical waiting times. Hill Climbing cannot determine which direction leads to improvement and may stop or wander randomly.

3. Ridge

A ridge occurs when improvement requires moving in multiple directions simultaneously.

Real-world Example:
Reducing congestion requires coordinated timing changes at several consecutive intersections. Changing only one signal does not improve traffic flow, causing Hill Climbing to fail.

iii. Simulated Annealing and Genetic Algorithms

Simulated Annealing

Simulated Annealing improves Hill Climbing by occasionally accepting worse solutions during the early stages of the search.

Working

- Starts with a high "temperature."
- Sometimes accepts poorer signal timings to escape local maxima.
- Gradually lowers the temperature.
- Eventually behaves like Hill Climbing and converges to a good solution.

Advantages

- Escapes local maxima.
- Handles plateaus and ridges effectively.
- Produces near-optimal solutions.
- Well suited for dynamic optimization.

Genetic Algorithm (GA)

Genetic Algorithms imitate natural evolution.

Working

1. Generate many different traffic signal plans.
2. Evaluate each plan using the fitness function.
3. Select the best-performing plans.
4. Apply crossover to combine good features.
5. Apply mutation to introduce new variations.
6. Repeat until high-quality solutions are obtained.

Advantages

- Explores many regions of the search space simultaneously.
- Avoids getting trapped in local maxima.
- Handles large and complex optimization problems.
- Easily adapts to changing traffic conditions.
- Highly scalable for smart cities.

iv. Recommended Approach

Recommended Algorithm: Genetic Algorithm

Justification

Criteria	Hill Climbing	Simulated Annealing	Genetic Algorithm
Search Speed	High	Moderate	Moderate
Escapes Local Maxima	No	Yes	Yes
Handles Plateaus	Poor	Good	Excellent
Handles Ridges	Poor	Good	Excellent
Scalability	Low	Moderate	High
Adaptability	Low	Good	Excellent
Global Optimization	No	Near Optimal	Near Optimal/Optimal

Reason

A smart city contains hundreds of intersections with continuously changing traffic conditions. The optimization problem is extremely large and dynamic.

- **Hill Climbing** is fast but easily gets stuck in local maxima.
- **Simulated Annealing** performs better by escaping local optima but explores one solution at a time.
- **Genetic Algorithms** evaluate many candidate solutions simultaneously, making them highly effective for large-scale, real-time traffic optimization.

Conclusion

For optimizing traffic signals in a smart city, **Genetic Algorithms are the most suitable approach**. They efficiently explore large search spaces, avoid local maxima, adapt to changing traffic patterns, and provide scalable, high-quality solutions. Although Hill Climbing is simple and fast, it is unsuitable for complex dynamic environments. Simulated Annealing improves upon Hill Climbing, but Genetic Algorithms offer the best balance of scalability, adaptability, and optimization performance for city-wide traffic management.

3. **Scenario:** An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

i. Explain why this problem requires an online search agent instead of offline search
ii. Analyze the challenges of partial observability and dynamic environments
iii. Describe how the rover builds and updates its internal model
iv. Compare online search with uninformed and informed search strategies
v. Justify the most suitable approach considering uncertainty, adaptability, and safety

i. Why This Problem Requires an Online Search Agent Instead of Offline Search

An **offline search agent** assumes that the complete environment is known before planning a path. It computes the entire route first and then executes it.

In the Mars rover scenario, this is not possible because:

- The rover has **no complete map** of the terrain.
- It discovers obstacles such as rocks and craters only while moving.
- The environment is only **partially observable** due to limited sensors.
- Decisions must be made in **real time** as new information becomes available.

Therefore, the rover requires an **online search agent**, which plans and acts simultaneously. It explores the environment, gathers new information, and updates its route while moving.

ii. Challenges of Partial Observability and Dynamic Environments

1. Partial Observability

The rover can sense only nearby areas, not the entire Martian surface.

Challenges:

- Unknown obstacles may exist beyond sensor range.
- The rover cannot predict all hazards in advance.
- Navigation decisions are based on incomplete information.
- The rover may need to change direction frequently.

Example:
A large crater is hidden behind a hill and becomes visible only when the rover approaches it.

2. Dynamic Environment

Although Mars changes more slowly than Earth, conditions can still vary during exploration.

Challenges:

- Dust storms may reduce visibility.
- Loose sand may make movement difficult.
- New hazards may be detected as the rover moves.
- Previously safe paths may become risky.

Example:
A path that appeared safe initially becomes dangerous after the rover detects soft sand that could trap its wheels.

iii. How the Rover Builds and Updates Its Internal Model

The rover maintains an **internal model** of its surroundings.

Initial Stage

- Starts with little or no map.
- Knows only its current location and mission objective.

Exploration

As the rover moves:

- Sensors detect nearby rocks, craters, slopes, and terrain.
- Newly discovered information is stored in memory.
- Unknown areas gradually become mapped.

Model Update

After each movement:

- Add newly discovered obstacles.
- Mark explored and unexplored regions.
- Update movement costs based on terrain difficulty.
- Recalculate the safest route if necessary.

Thus, the internal model continuously improves throughout the mission, enabling better future decisions.

iv. Comparison of Online Search with Uninformed and Informed Search

Feature	Online Search	Uninformed Search	Informed Search
Complete map required	No	Yes	Usually Yes
Works with unknown environments	Yes	No	Limited
Uses heuristic information	Can use heuristics	No	Yes
Real-time decision making	Yes	No	Limited
Adaptability	Excellent	Poor	Moderate
Suitable for Mars rover	Yes	No	Partially

Analysis

Uninformed Search (BFS, DFS):

- Assumes the environment is known.
- Cannot efficiently explore unknown terrain.
- Not suitable for real-time exploration.

Informed Search (A*):

- Uses heuristic information to find efficient paths.
- Performs well when a map is available.
- Needs replanning whenever new obstacles are discovered.

Online Search:

- Explores and plans simultaneously.
- Learns about the environment while moving.
- Continuously updates decisions based on new sensor data.

v. Most Suitable Approach

Recommended Approach: Online Search Agent (Model-Based)

Justification

Criterion	Online Search
Handles uncertainty	Excellent
Adapts to new information	Excellent
Operates without a complete map	Yes
Supports real-time decisions	Yes
Safety	High
Suitable for Mars exploration	Excellent

Reason

The Mars rover operates in an **unknown and partially observable environment**, making offline planning impractical. An online search agent:

- Explores the terrain while navigating.
- Updates its internal model with sensor data.
- Avoids newly discovered hazards.
- Replans routes whenever conditions change.
- Ensures safe and efficient sample collection despite uncertainty.

Conclusion

For autonomous Mars exploration, an **online search agent with a continuously updated internal model** is the most suitable approach. Unlike offline, uninformed, or traditional informed search algorithms, online search can operate without a complete map, adapt to newly discovered hazards, and make safe real-time navigation decisions. This makes it ideal for uncertain and dynamic environments such as the Martian surface.

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints:No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

i. Clearly define variables, domains, and constraints with explanation
ii. Explain how backtracking search works in this scenario
iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency
iv. Analyze real-world challenges and justify the best strategy for scalability

i.Variables, Domains, and Constraints

1. Variables

Each exam is treated as a variable.

Examples:

- E1 = Mathematics Exam
- E2 = Physics Exam
- E3 = Chemistry Exam
- E4 = Computer Science Exam

Each variable requires an assigned **time slot** and **exam hall**.

2. Domains

The domain of each variable is the set of possible values.

For every exam:

- Available time slots
  - Morning Session
  - Afternoon Session
  - Evening Session
- Available exam halls
  - Hall A
  - Hall B
  - Hall C

Thus, each exam can be assigned any valid combination of time slot and hall.

3. Constraints

The timetable must satisfy the following constraints:

a) No Student Overlapping Exams

A student enrolled in two subjects cannot have both exams scheduled at the same time.

Example:
If a student takes Mathematics and Physics, these exams must be held in different time slots.

b) Limited Exam Halls

The number of exams conducted simultaneously cannot exceed the available exam halls.

Example:
If only three halls are available, at most three exams can occur at the same time.

c) Subject Ordering Constraints

Some exams must be conducted before others.

Example:
Programming Fundamentals must be completed before Advanced Programming.

d) Faculty Availability

An exam can only be scheduled when the assigned faculty member is available.

e) Special Student Accommodations

Students requiring extra time or special facilities must be assigned suitable halls and schedules.

ii. Backtracking Search

Working Principle

Backtracking is a depth-first search algorithm that assigns values one variable at a time.

Steps

1. Select an unscheduled exam.
2. Assign an available time slot and hall.
3. Check whether all constraints are satisfied.
4. If valid, continue scheduling the next exam.
5. If a conflict occurs, undo the assignment (backtrack).
6. Try another available assignment.
7. Continue until all exams are successfully scheduled.

Example

- Mathematics → Monday Morning ✓
- Physics → Monday Morning ✗ (same students)

Conflict detected.

Backtracking:

- Move Physics → Monday Afternoon ✓

Continue with remaining exams.

Advantages

- Guarantees a valid timetable if one exists.
- Systematically explores possible assignments.
- Simple and effective for CSP problems.

Limitations

- May become slow as the number of courses and students increases.
- Large search spaces can require extensive backtracking.

iii. Constraint Propagation (Forward Checking)

Forward Checking

Forward checking improves backtracking by removing invalid choices before they are explored.

Working

After assigning an exam:

- Remove conflicting time slots from the domains of related exams.
- If any exam has no remaining valid time slot, immediately backtrack.

Example

Suppose:

- Mathematics is scheduled on Monday Morning.
- Many students also take Physics.

Forward checking removes Monday Morning from Physics' possible time slots.

Instead of exploring an impossible schedule later, the conflict is detected immediately.

Benefits

- Detects conflicts early.
- Reduces unnecessary search.
- Decreases the number of backtracking steps.
- Speeds up timetable generation.
- Improves scalability for large universities.

iv. Real-World Challenges and Best Strategy

Real-World Challenges

1. Large Number of Courses

- Hundreds of exams must be scheduled.

2. Large Student Population

- Thousands of students create numerous overlap constraints.

3. Faculty Availability

- Faculty members may have limited availability.

4. Limited Resources

- Exam halls and invigilators are limited.

5. Special Requirements

- Some students require extra time or accessible rooms.

6. Last-Minute Changes

- Faculty absence or hall unavailability may require timetable updates.

Best Strategy for Scalability

Recommended Approach: Backtracking Search with Constraint Propagation (Forward Checking)

Justification

Feature	Simple Backtracking	Backtracking + Forward Checking
Finds valid timetable	Yes	Yes
Detects conflicts early	No	Yes
Number of backtracking steps	High	Low
Execution speed	Moderate	Fast
Scalability	Moderate	High
Suitable for large universities	Limited	Excellent

Reason

Backtracking alone guarantees a correct timetable but may waste time exploring invalid assignments. Forward checking eliminates impossible choices early, significantly reducing the search space. This combination efficiently handles multiple constraints, adapts to large numbers of courses and students, and scales well for real-world university scheduling systems.

Conclusion

The university exam scheduling problem is naturally modeled as a **Constraint Satisfaction Problem (CSP)**, where exams are variables, available time slots and halls are domains, and scheduling rules are constraints. **Backtracking search** systematically constructs a valid

timetable, while **constraint propagation using forward checking** improves efficiency by detecting conflicts early and reducing unnecessary search. Together, these techniques provide a scalable, reliable, and practical solution for generating exam timetables in large universities.

5. **Scenario:** Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning)A gaming company is developing an AI opponent for a real-time strategy game. The AI must complete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

i. Explain how the Minimax algorithm models this problem

ii. Analyze the computational challenges of large game trees

iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning

iv. Design an evaluation function and justify the factors considered

v. Recommend a strategy for balancing optimality and real-time performance

i. How the Minimax Algorithm Models This Problem

The **Minimax algorithm** is an adversarial search algorithm used in two-player games where one player (the AI) tries to maximize its advantage while the opponent tries to minimize it.

Modeling the Problem

- **Initial State:** Current game position.
- **Players:**
  - **MAX:** AI player (tries to maximize the score).
  - **MIN:** Human opponent (tries to minimize the AI's score).
- **Actions:** All possible legal moves from the current game state.
- **State Transition:** Applying a move generates a new game state.
- **Terminal State:** Win, loss, draw, or search depth limit.
- **Utility Function:** Assigns a numerical value to terminal states.

Working

1. The AI generates all possible moves.
2. It predicts the opponent's best possible response.
3. This process continues until a terminal state or depth limit is reached.
4. Utility values are propagated back up the tree.
5. The AI selects the move with the highest Minimax value.

Example:

- AI considers three possible attacks.
- For each attack, it predicts the opponent's strongest defense.
- The AI chooses the move that provides the best outcome even against the opponent's optimal play.

ii. Computational Challenges of Large Game Trees

Real-time strategy games have an enormous number of possible game states.

Challenges

1. Large Branching Factor

Each game state may have many legal moves.

Example:
20 possible moves at each turn create millions of game states after only a few levels.

2. Deep Search Trees

Games often last many turns.

The number of nodes grows exponentially:

O(bd)O(b^d)O(bd)

where:

- **b** = branching factor
- **d** = search depth

3. Real-Time Constraints

The AI must decide within milliseconds.

Searching the complete game tree is impossible during gameplay.

4. Memory Consumption

Storing large search trees requires significant memory.

5. Dynamic Game Environment

The game state changes continuously due to player actions, requiring frequent replanning.

iii. How Alpha-Beta Pruning Improves Efficiency

Working Principle

Alpha-Beta pruning is an optimization of the Minimax algorithm.

It eliminates branches that cannot influence the final decision, reducing the number of nodes evaluated.

Parameters

- **Alpha (α):** Best value found so far for the MAX player.
- **Beta (β):** Best value found so far for the MIN player.

Pruning Rule

Whenever:

α≥β\alpha \ge \betaα≥β

the remaining branches are ignored because they cannot improve the final decision.

Example

Suppose:

- AI already has a move worth **+8**.
- Another branch guarantees at most **+5**.

Since the second branch cannot outperform the current best move, it is pruned without further evaluation.

Advantages

- Reduces the number of nodes explored.
- Speeds up decision making.
- Allows deeper searches within the same time limit.
- Produces the same optimal result as Minimax when node ordering is good.

iv. Evaluation Function

Since searching the entire game tree is impractical, the AI evaluates non-terminal states using an evaluation function.

Example Evaluation Function

Score=5(Resources)+4(Army Strength)+3(Map Control)+2(Health)+1(Technology)−4(Enemy Advantage)\text{Score} = 5(\text{Resources}) +4(\text{Army Strength}) +3(\text{Map Control}) +2(\text{Health}) +1(\text{Technology}) -4(\text{Enemy Advantage})Score=5(Resources)+4(Army Strength)+3(Map Control)+2(Health)+1(Technology)−4(Enemy Advantage)

Factors Considered

1. Resources

More resources enable stronger units and faster expansion.

2. Army Strength

A larger and stronger army increases the likelihood of victory.

3. Map Control

Controlling strategic areas provides tactical and economic advantages.

4. Unit Health

Healthier units improve combat effectiveness and survival.

5. Technology Level

Advanced technologies unlock stronger abilities and units.

6. Enemy Advantage

The evaluation penalizes states where the opponent has superior forces or strategic positions.

Justification

These factors directly influence the chances of winning and allow the AI to estimate the quality of intermediate game states when a full search is not feasible.

v. Recommended Strategy for Balancing Optimality and Real-Time Performance

Recommended Approach

Minimax + Alpha-Beta Pruning + Depth-Limited Search + Evaluation Function

Justification

Feature	Minimax	Minimax + Alpha-Beta
Optimal move	Yes	Yes
Search speed	Slow	Fast
Nodes evaluated	Very High	Much Lower
Real-time suitability	Poor	Excellent
Scalability	Limited	High

Why This Strategy?

- **Minimax** ensures rational decision-making by considering the opponent's best responses.
- **Alpha-Beta pruning** significantly reduces unnecessary computations while preserving the optimal result.
- **Depth-limited search** ensures decisions are made within strict time limits.
- **Evaluation functions** estimate the quality of non-terminal game states, enabling effective decisions without exploring the full game tree.

This combination provides an excellent balance between decision quality and computational efficiency, making it well suited for real-time strategy games.

Conclusion

For a real-time strategy game, **Minimax with Alpha-Beta pruning, depth-limited search, and a well-designed evaluation function** is the most suitable approach. Minimax models the competition between the AI and the human player, while Alpha-Beta pruning greatly reduces the search effort without affecting optimality. By combining pruning with heuristic evaluation and limited search depth, the AI can make strong strategic decisions within strict time constraints, ensuring both high performance and responsiveness during gameplay.

RUBRICS FOR CALCULATION

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Max Marks
Understanding of Scenario	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario	10
Identification of Key Issues	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues	10
Application of Concepts	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts	12.5
Decision Making	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided	10
Clarity of Response	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand	7.5
				Total	50